BT5110 Tutorial 1

Case: NUN Student Book Exchange

Students at the National University of Ngendipura (NUN) buy books for their studies. They also lend and borrow books among themselves. NUN Students Association (NUNStA) has commissioned Apasaja Private Limited to implement an online book exchange system that records information about students and the books they own, borrow and lend.

For each student, the database records the student's name, faculty, department, university join date, and graduation date. Each student is uniquely identified by email address.

For each book, the database records the title, format, number of pages, language, authors, publisher, publication date, and the industry-standard book identifiers ISBN10 and ISBN13. It is possible that a book may exist in the database even when no current student owns a copy because the owner has graduated or a lecturer has recommended the book before any student has purchased it.

A loan record stores the date at which a book copy is borrowed and the date at which it is returned. For auditing purposes, information about books, copies, and students is retained while it is referenced by existing copy or loan records.

1. Create and Populate the Database

Download the following SQL files from Canvas: Files → Cases → Book Exchange.

`NUNStASchema.sql`	`NUNStAStudent.sql`	`NUNStAClean.sql`
`NUNStABook.sql`	`NUNStACopy.sql`	`NUNStALoan.sql`

(a) For each file, briefly explain what it does and identify the main SQL statements used.

(b) Use the files to create and populate the database. If you encounter an error, identify the cause, fix it, and explain your correction.

(c) Identify the primary key of each relation.

(d) Identify all foreign keys. State the referencing relation and the referenced relation.

(e) Which relations have composite primary keys? Why is more than one attribute needed?

(f) Find examples of UNIQUE, NOT NULL, and CHECK constraints. What business rule does each constraint enforce?

2. Insert, Update and Delete

Perform parts (a) to (g) in sequence. For each operation, first predict whether it will succeed or fail and explain why using the relevant constraint or SQL behavior. Then execute the statement to check your answer.

(a) Insert the following book.

```
INSERT INTO book VALUES (
  'An Introduction to Database Systems',
  'paperback',
  640,
  'English',
  'C. J. Date',
  'Pearson',
  '2003-01-01',
  '0321197844',
  '978-0321197849'
);
```

(b) Repeat the insertion but change ISBN13 to '978-0201385908'.

(c) Repeat the insertion but change ISBN10 to '0201385902'.

(d) Insert a new student by explicitly naming the target columns. What value is stored in any omitted column?

```
INSERT INTO student (email, name, join, faculty, department) VALUES (
  'ben.lim@u.nun.edu',
  'BEN LIM',
  '2026-08-10',
  'School of Computing',
  'CS'
);
```

(e) Change the department name from 'CS' to 'Computer Science'. How many rows are affected?

```
UPDATE student
SET department = 'Computer Science'
WHERE department = 'CS';
```

(f) Delete all students whose department is 'chemistry'.

(g) Delete all students whose department is 'Chemistry'. How many rows match the condition? Does the deletion succeed? Compare the result with part (f).

3. Simplifying the Schema

(a) The copy table contains an available attribute. Explain why this value can be derived from existing loan data instead of being stored separately. What information would you check to determine whether a copy is currently available?

(b) Assume that each department belongs to exactly one faculty. The student table stores both department and faculty. Explain why repeatedly storing the faculty for every student leads to redundancy and may cause update anomalies. Sketch a simple revised schema that stores each department–faculty relationship only once.